

NPTEL Course on "Computer System Optimization for ML"

Lecture #26

Agenda

- Recap
- Acceleration of LLMs (contd.)
- Speculative Decoding

NPTEL

LinFormer (Reducing the Complexity to O(n))

- $head_i = Attention(QW_i^Q, E_i KW_i^K, F_i VW_i^V)$
  - E_i and F_i are $n \times k$ projection matrices where $k \ll n$
- $softmax\left(\frac{QW_i^Q (E_i KW_i^K)^T}{\sqrt{d_k}}\right) F_i VW_i^V$
- The dimension of the inner matrix is $n \times k$
- Computational complexity $O(nk) \cong O(n)$
  - $k = O\left(\frac{d}{\epsilon^2}\right)$
  - d is the embedding size
  - Independent of n

NPTEL

Results: Improvement in Efficiency

Time Saved

length n	projected dimensions k				
	128	256	512	1024	2048
512	1.5x	1.3x	-	-	-
1024	1.7x	1.6x	1.3x	-	-
2048	2.6x	2.4x	2.1x	1.3x	-
4096	3.4x	3.2x	2.8x	2.2x	1.3x
8192	5.5x	5.0x	4.4x	3.5x	2.1x
16384	8.6x	7.8x	7.0x	5.6x	3.3x
32768	13x	12x	11x	8.8x	5.0x
65536	20x	18x	16x	14x	7.9x

Memory Saved

length n	projected dimensions k				
	128	256	512	1024	2048
512	1.7x	1.5x	-	-	-
1024	3.0x	2.9x	1.8x	-	-
2048	6.1x	5.6x	3.6x	2.0x	-
4096	14x	13x	8.3x	4.3x	2.3x
8192	28x	26x	17x	8.5x	4.5x
16384	56x	48x	32x	16x	8x
32768	56x	48x	36x	18x	16x
65536	60x	52x	40x	20x	18x

NTL

Agenda

- Recap
- **Acceleration of LLMs (contd.)**
- Speculative Decoding

NPTEL

FlashAttention: Acceleration of Training

Standard attention implementation

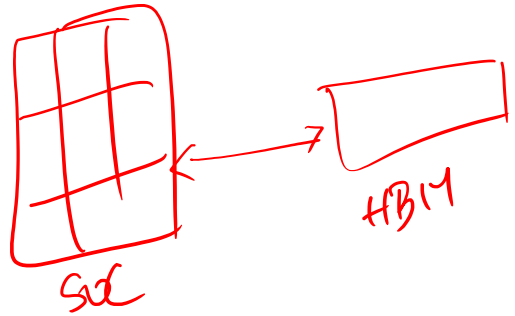
$$\boldsymbol{Attention}\,(\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V}) = \boldsymbol{softmax}\left(\frac{\boldsymbol{QK^T}}{\sqrt{\boldsymbol{d_k}}}\right)\boldsymbol{V}$$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
4: Return $\mathbf{O}$.

Lots of memory (HBM) access



NPTEL

FlashAttention: Tiling

- $\boldsymbol{Attention}\,(\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V}) = \boldsymbol{softmax}\left(\frac{\boldsymbol{QK^T}}{\sqrt{\boldsymbol{d_k}}}\right)\boldsymbol{V}$
- **Split Q,K,V into blocks**
- **Load them from slow HBM to faster SRAM**
- **Compute the attention output of the blocks**
- **Requires keeping track of extra statistics for softmax**



NPTEL

FlashAttention: Recomputation

- **Need to store S (=QK^T) and P (=softmax(S))**
  - Useful for backward pass
- **No need to actually store them**
- **We can recompute them**

NPTEL

FlashAttention: Algorithm

Algorithm 1 FlashAttention

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M.

1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
5: **for** $1 \le j \le T_c$ **do**
6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
7: **for** $1 \le i \le T_r$ **do**
8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
14: **end for**
15: **end for**
16: Return $\mathbf{O}$.

FlashAttention: Results

Forward + backward runtime of standard attention and FlashAttention for GPT-2 medium (seq. length 1024, head dim. 64, 16 threads, batch size 64) on A100 GPU

Attention	Standard	FlashAttention
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3

NPTEL

Agenda

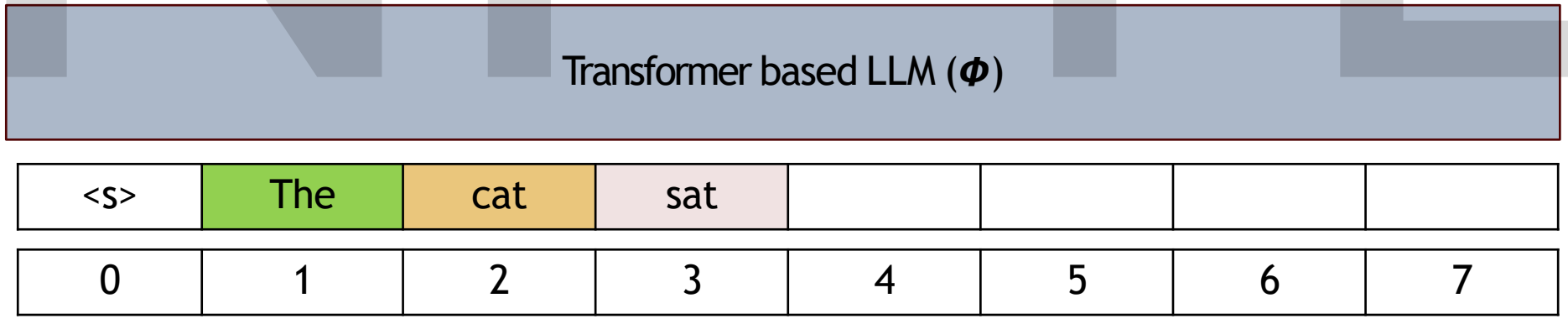
- Recap
- Acceleration of LLMs (contd.)
- **Speculative Decoding**

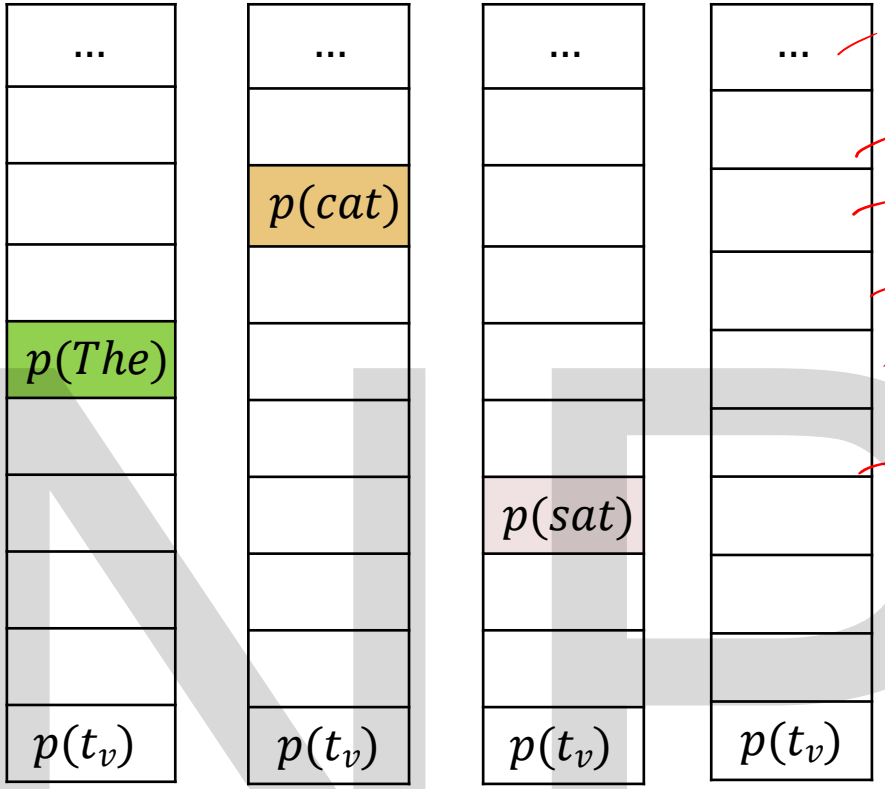
NPTEL

LLM Inference

NPTEL

Forward Pass (#1) : Prefill

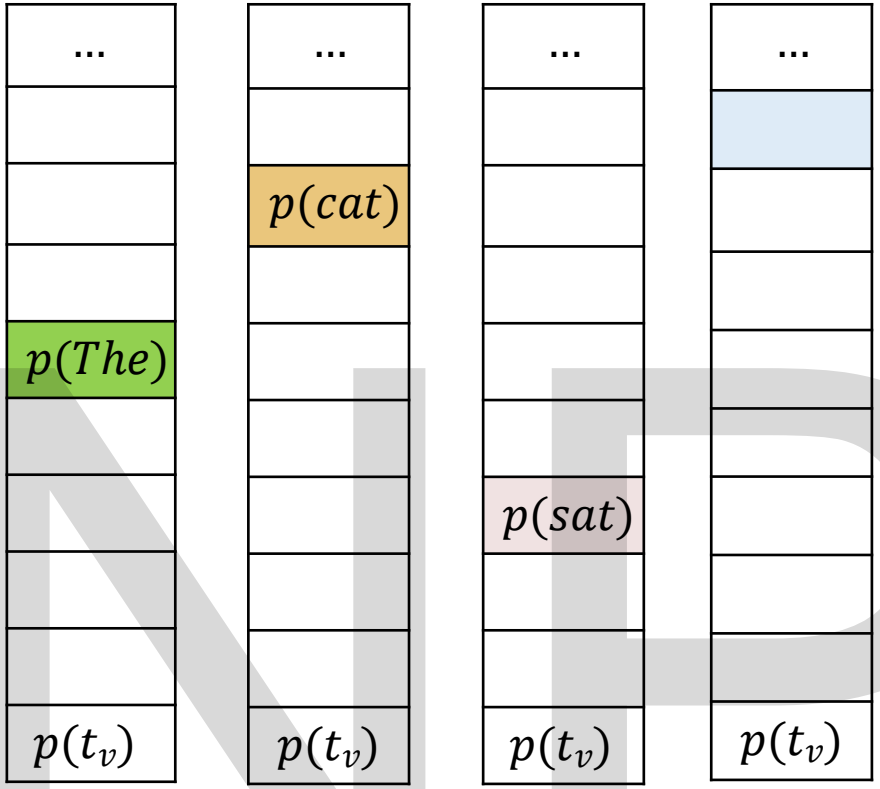




Prob. Dist. at all steps

Transformer based LLM (Φ)

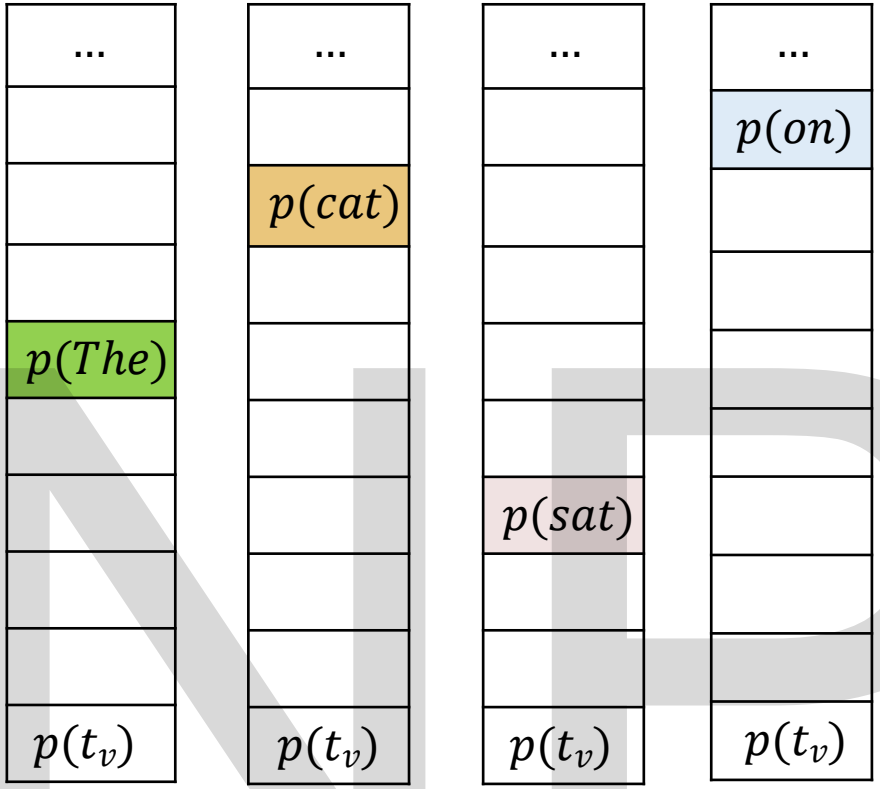
<s>	The	cat	sat				
0	1	2	3	4	5	6	7



Pick the token having max. probability at step 3

Transformer based LLM (Φ)

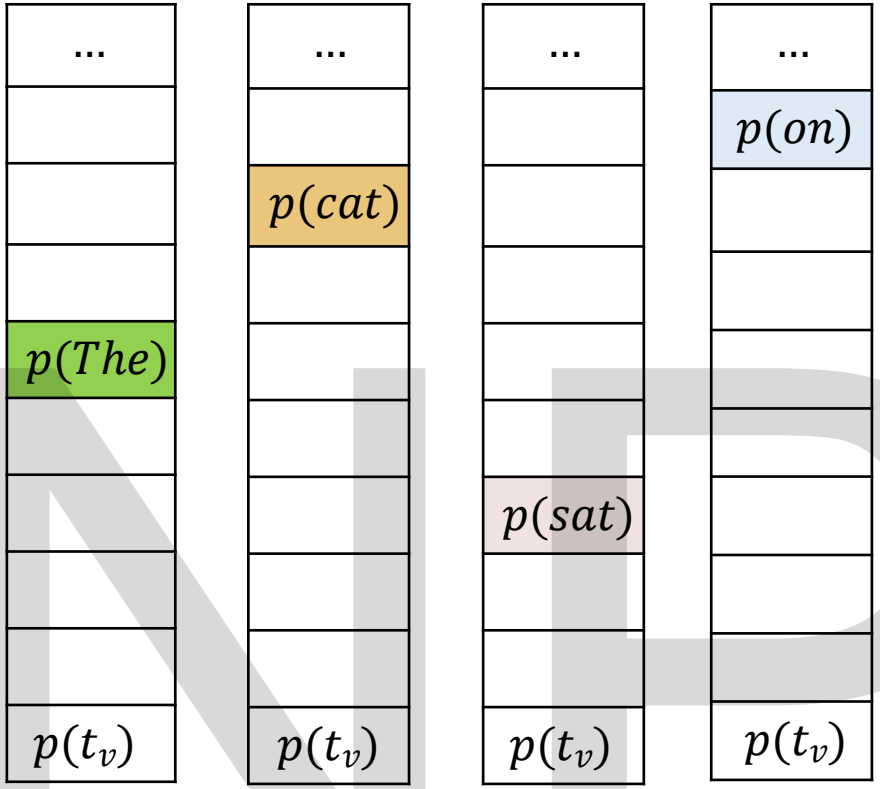
<s>	The	cat	sat				
0	1	2	3	4	5	6	7



Pick the token having max. probability at step 3

Transformer based LLM (Φ)

<s>	The	cat	sat				
0	1	2	3	4	5	6	7



Fill at step 4

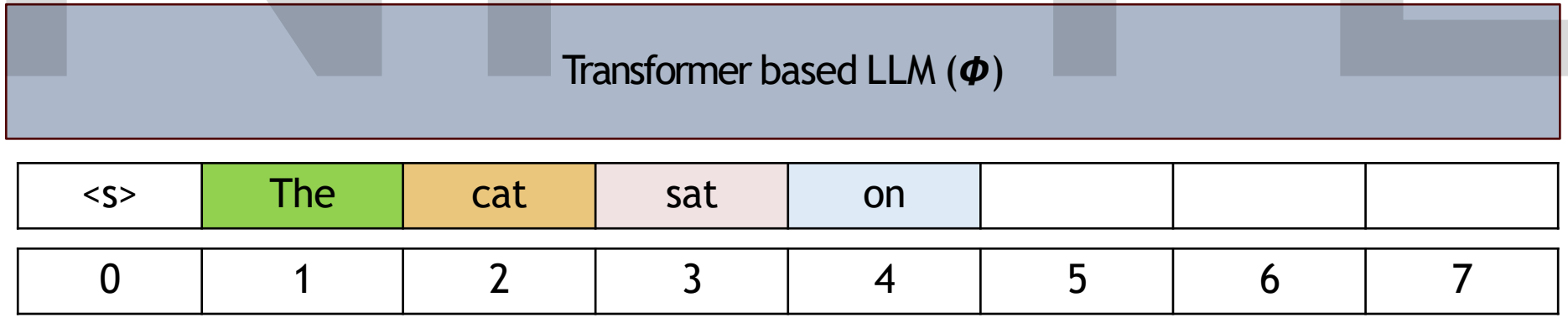
Transformer based LLM (Φ)

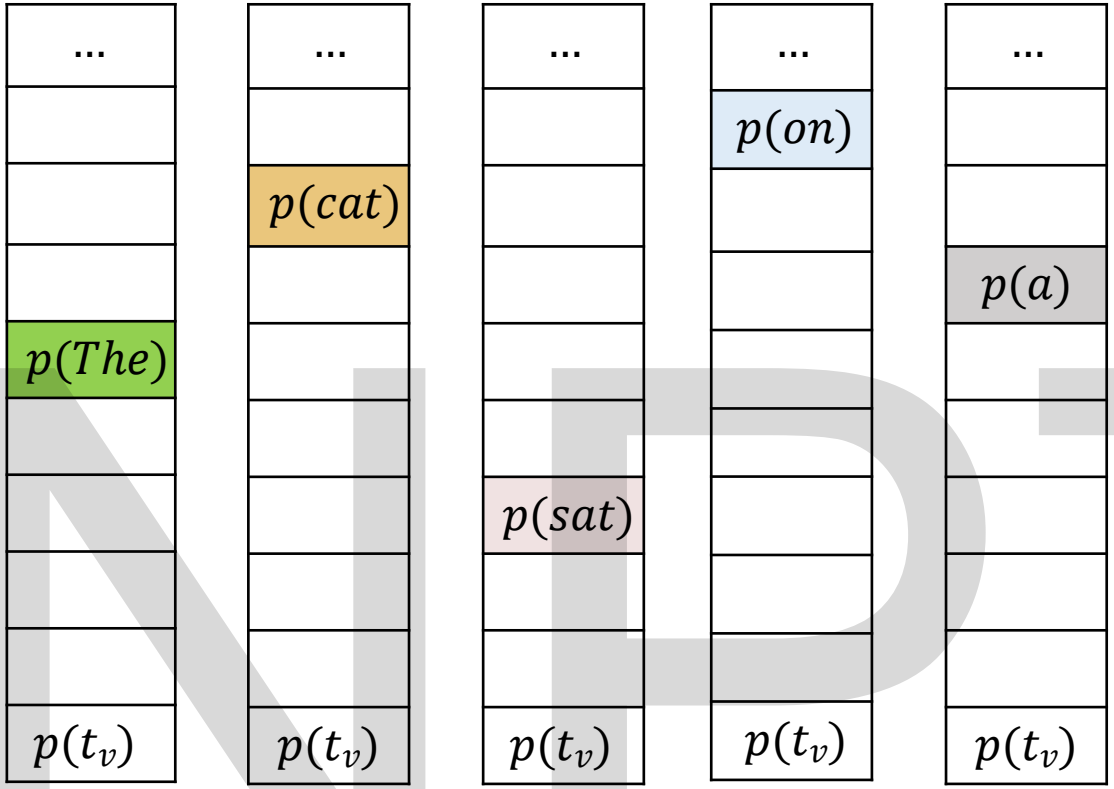
<s>	The	cat	sat	on			
0	1	2	3	4	5	6	7

through an LLM

NPTEL

Fwd. Pass (#2)

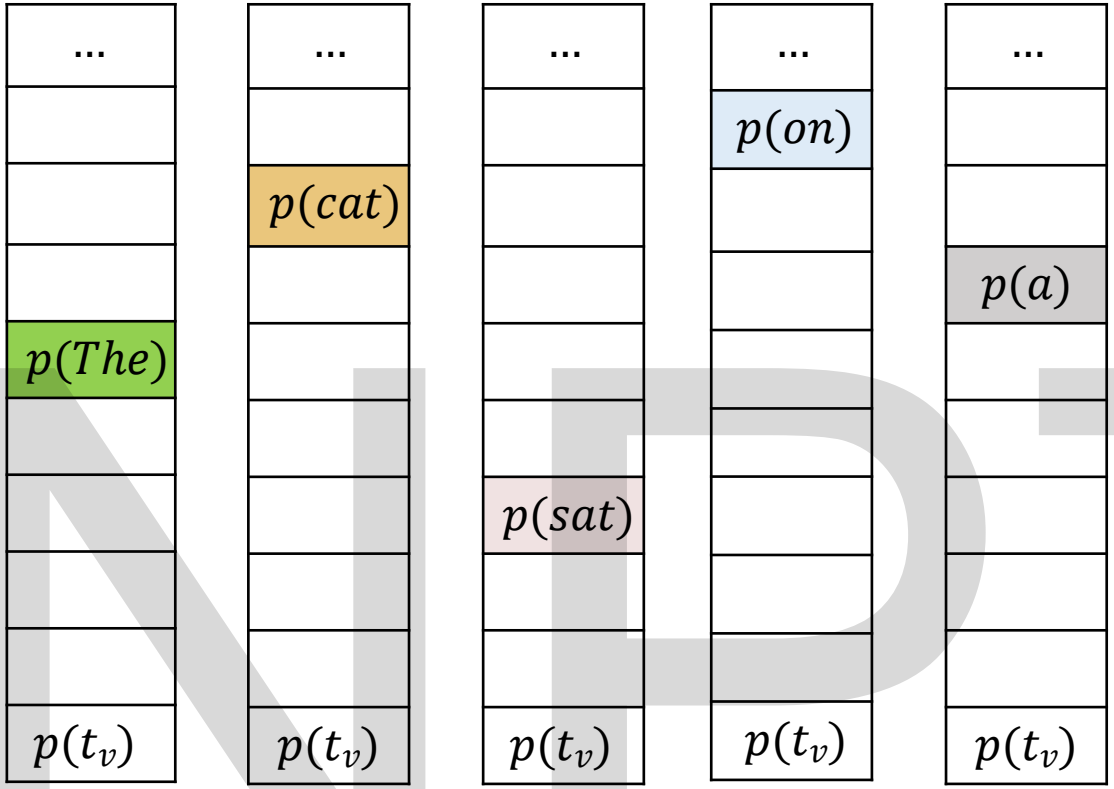




Fwd. pass (#2) to get distribution at step 4

Transformer based LLM (Φ)

<s>	The	cat	sat	on			
0	1	2	3	4	5	6	7



Fill at step 5

Transformer based LLM (Φ)

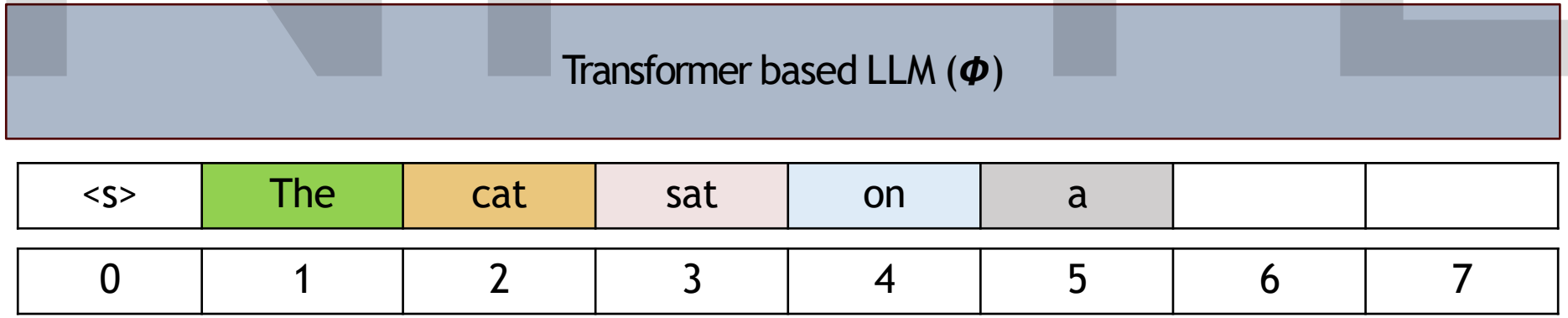
<s>	The	cat	sat	on	a		
0	1	2	3	4	5	6	7

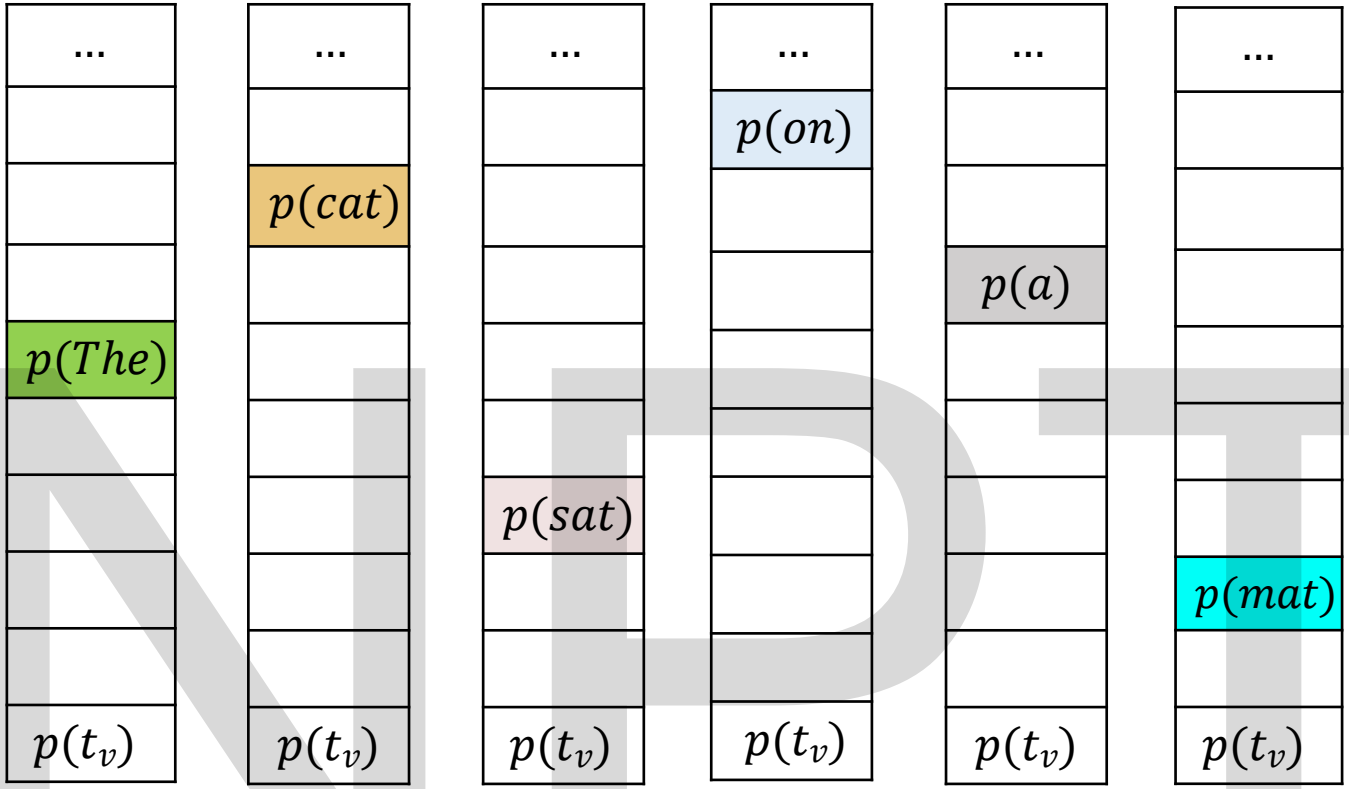
NPTEL

rough an LLM

NPTEL

Fwd. pass again (#3)

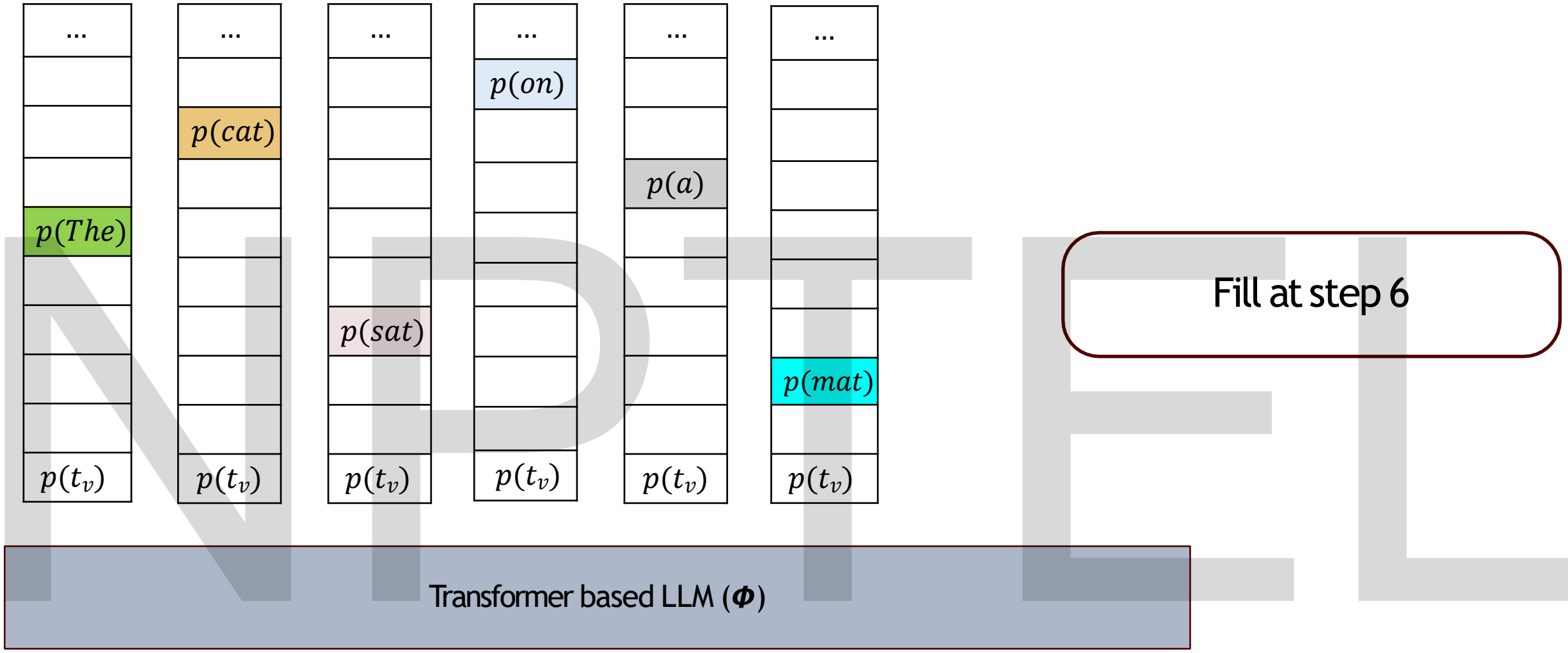




Fwd. pass again (#3)

Transformer based LLM (Φ)

<s>	The	cat	sat	on	a		
0	1	2	3	4	5	6	7



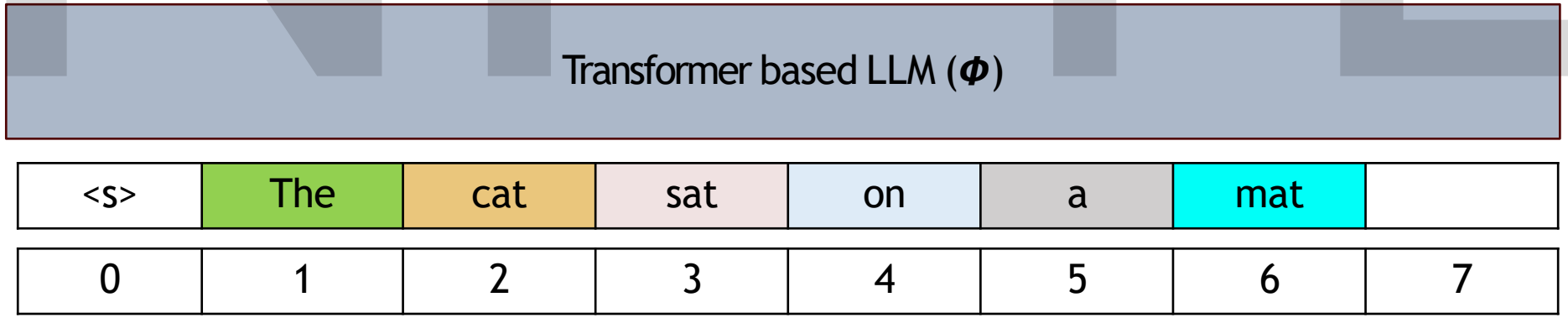
Fill at step 6

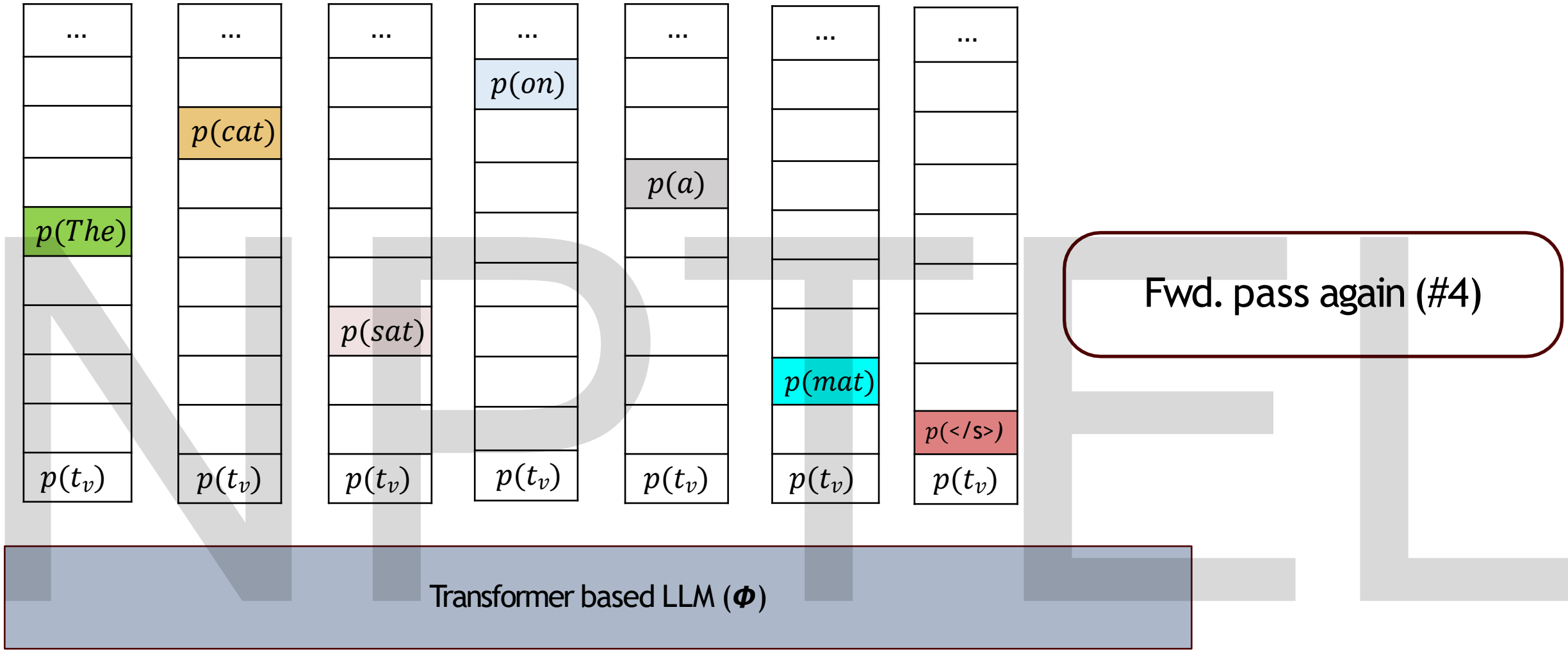
NPTEL

an LLM

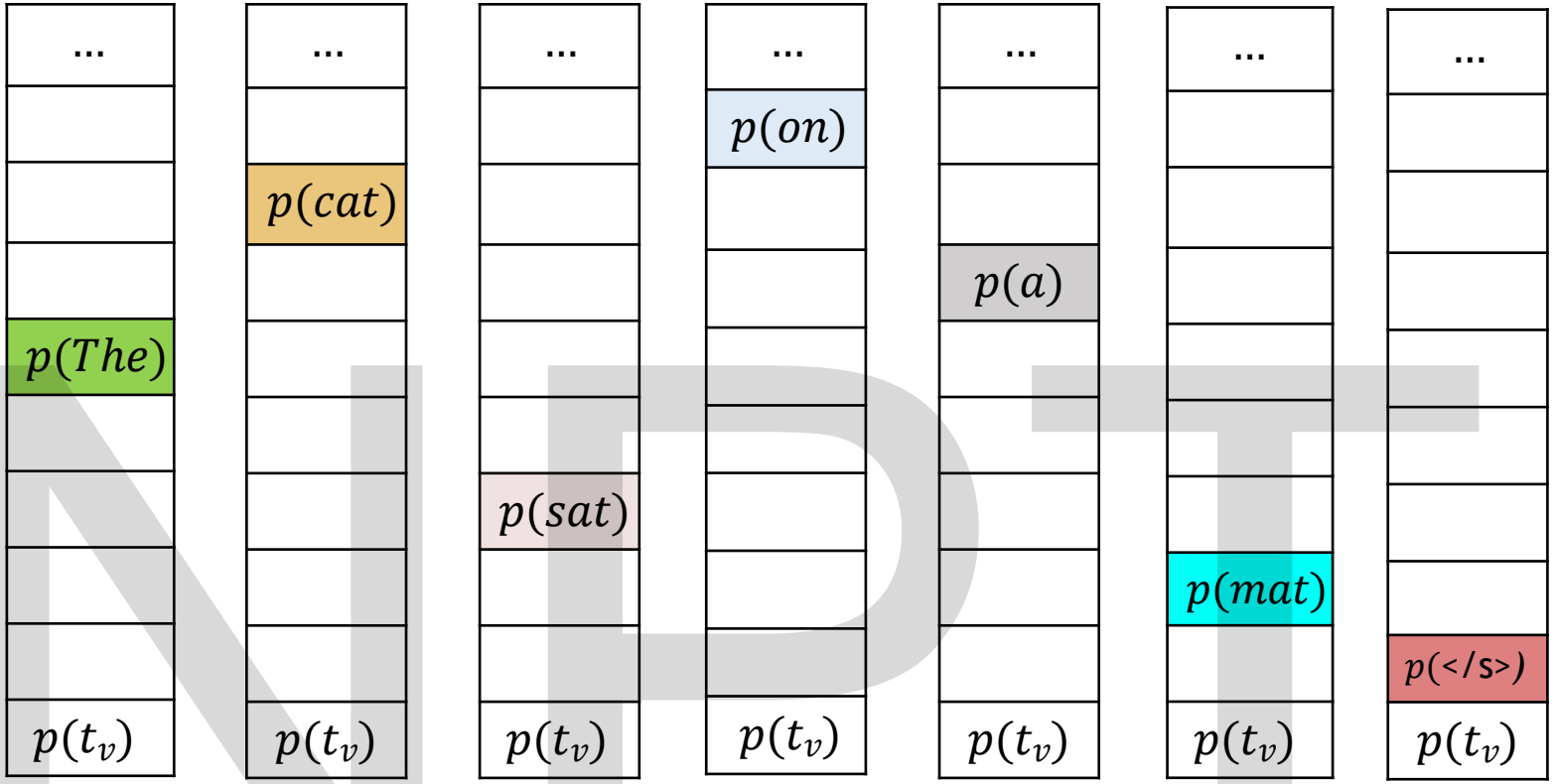
NPTEL

Fwd. pass again (#4)





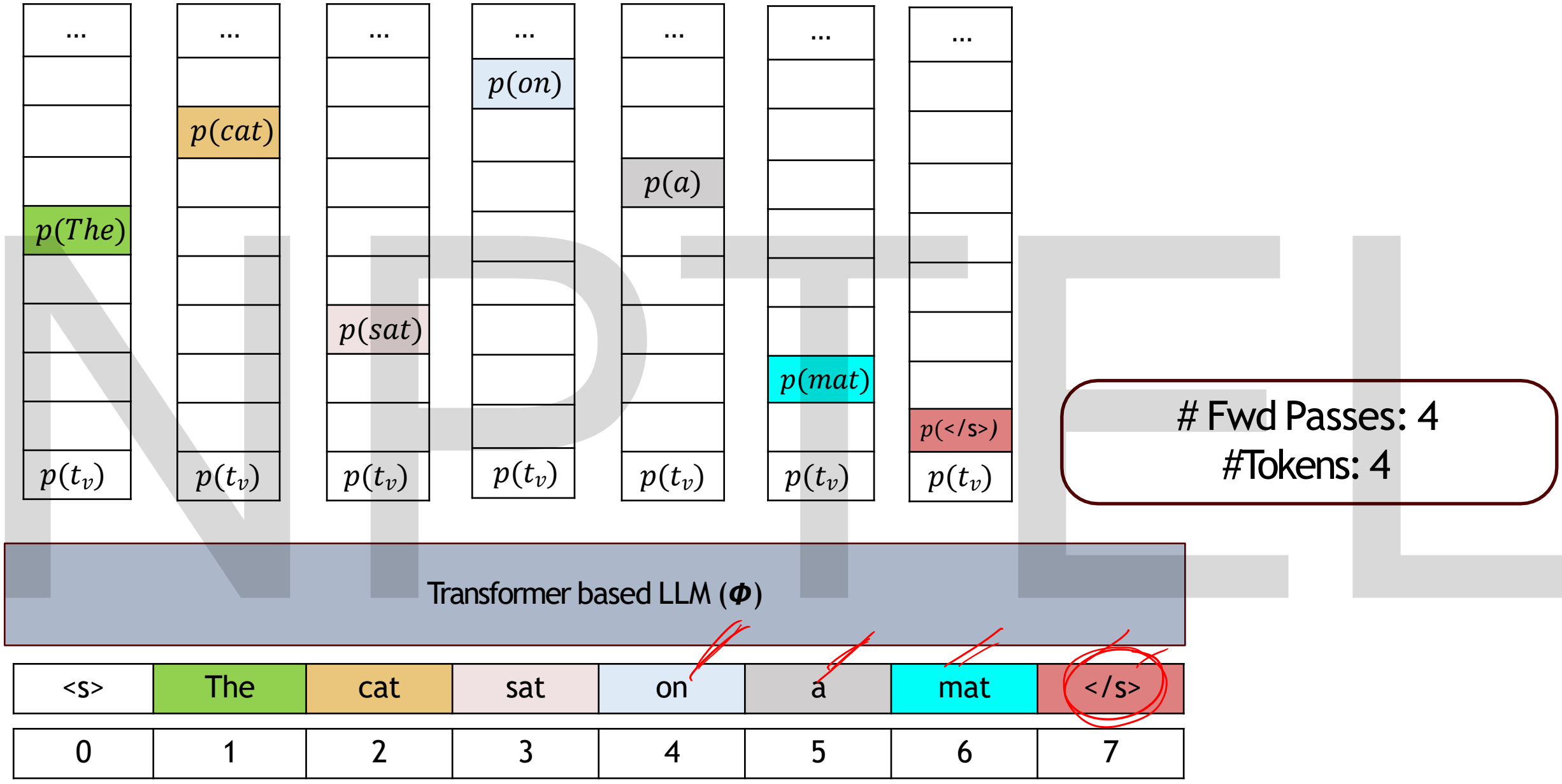
Fwd. pass again (#4)



Stop at end of seq. token: </s>

Transformer based LLM (Φ)

<s>	The	cat	sat	on	a	mat	</s>
0	1	2	3	4	5	6	7


...
p(The)
p(cat)
p(sat)
p(on)
p(a)
p(mat)
p(</s>)
p(t_v)
Transformer based LLM (Φ)
<s>
The
cat
sat
on
a
mat
</s>
0
1
2
3
4
5
6
7
Fwd Passes: 4
#Tokens: 4

For Autoregressive Models...

Decoding K tokens takes K **serial runs**.

Can we somehow decode several tokens **in parallel**?

NPTEL

Inference through an LLM

Can we use a guess output to speed up inference?

- Input prompt: *"The cat sat"*

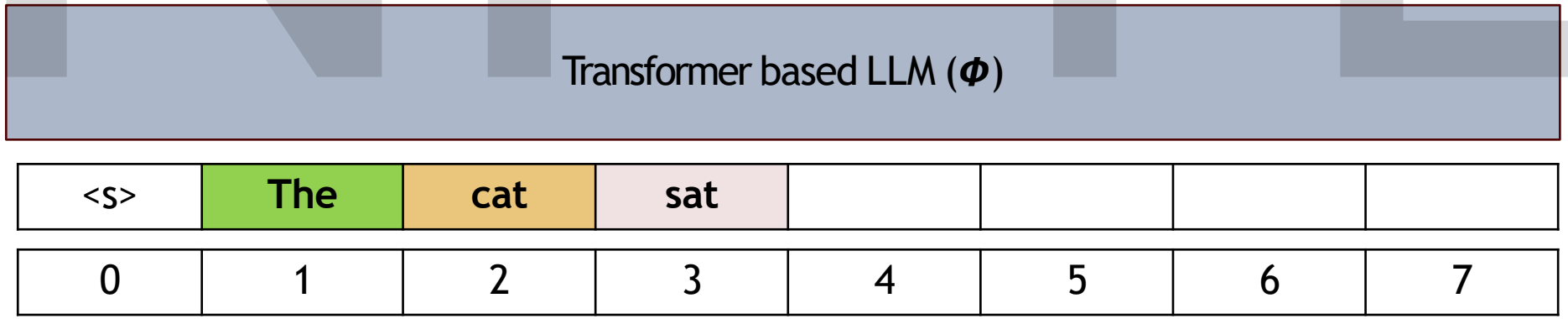
Transformer based LLM (Φ)

<s>	The	cat	sat				
0	1	2	3	4	5	6	7

Inference through an LLM

NPTEL

- **Input prompt:** "*The cat sat*"
- **Guess:** "*on the chair*"

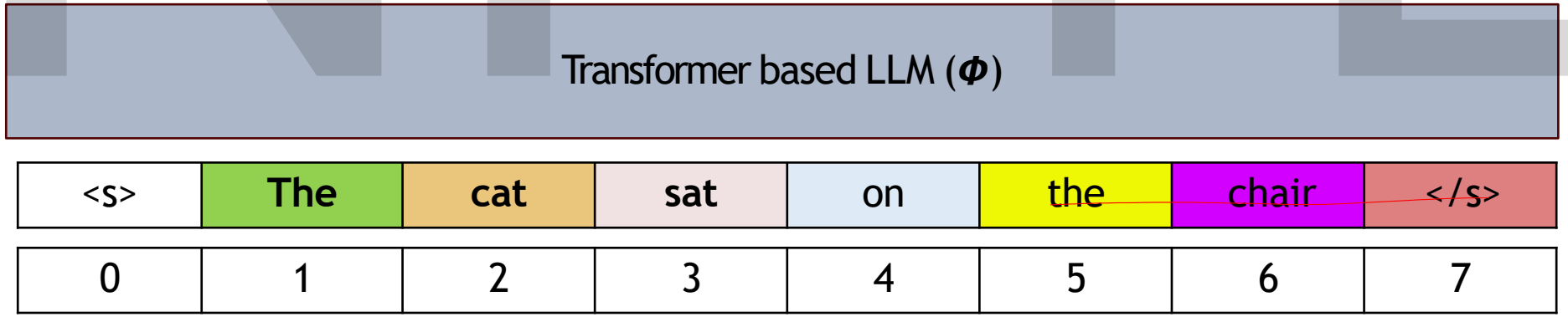


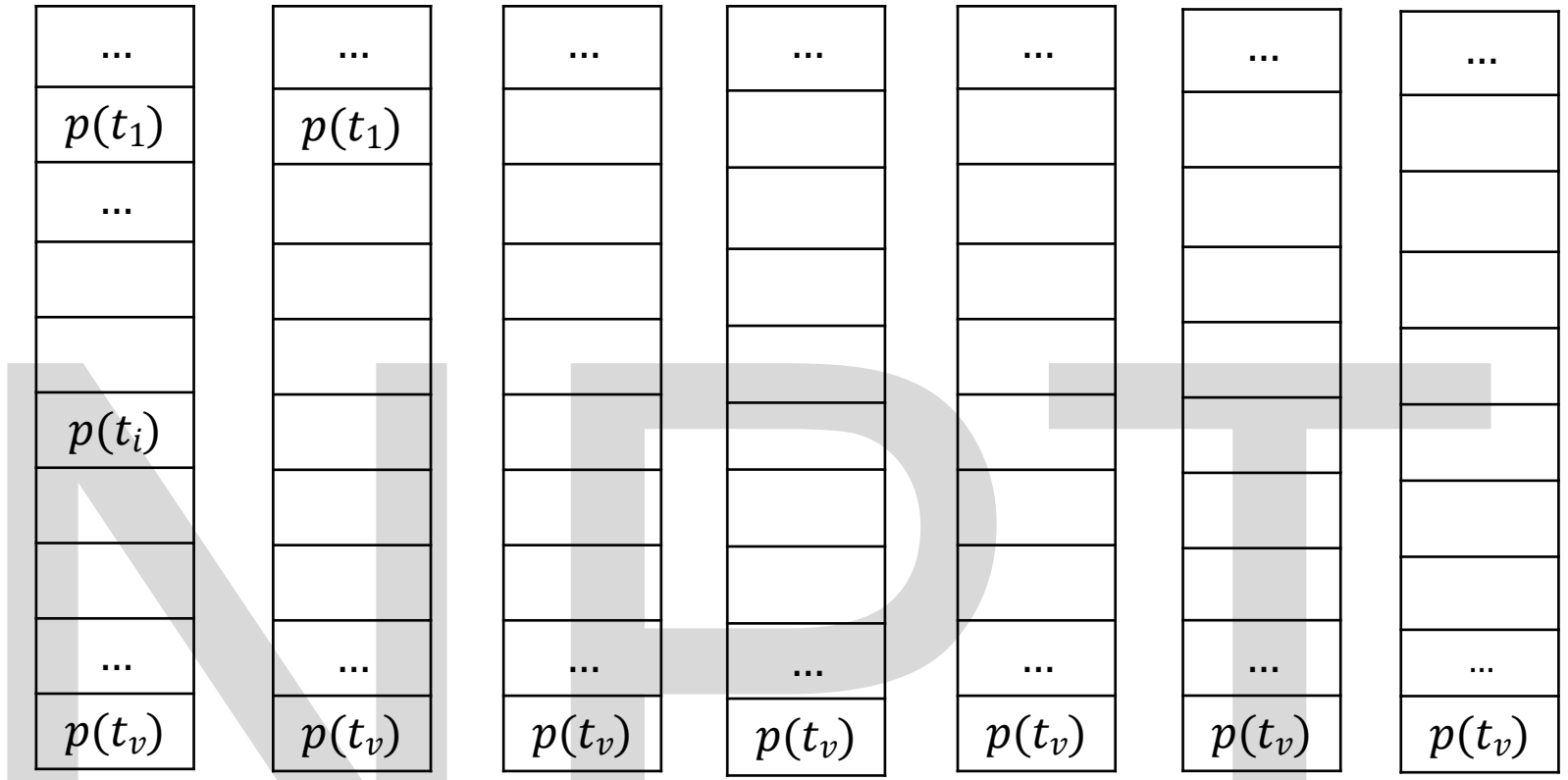
Inference through an LLM

NPTEL

- **Input prompt:** *"The cat sat"*
- **Guess:** *"on the chair </s>"*

Run a forward pass with the guess completion



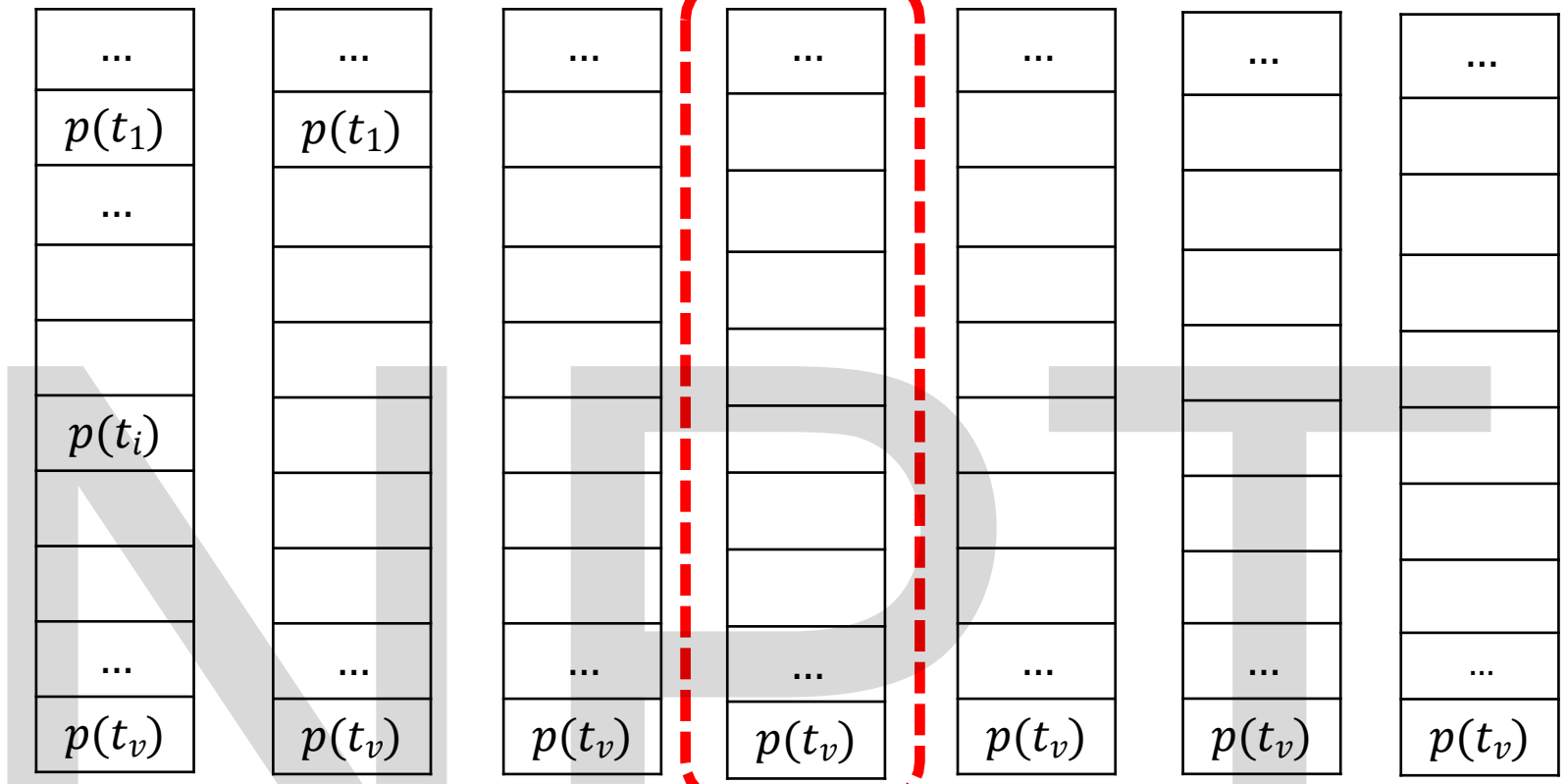


- **Input prompt:** *"The cat sat"*
- **Guess:** *"on the chair </s>"*

We get prob. dist. at each step

Transformer based LLM (Φ)

<s>	The	cat	sat	on	the	chair	</s>
0	1	2	3	4	5	6	7

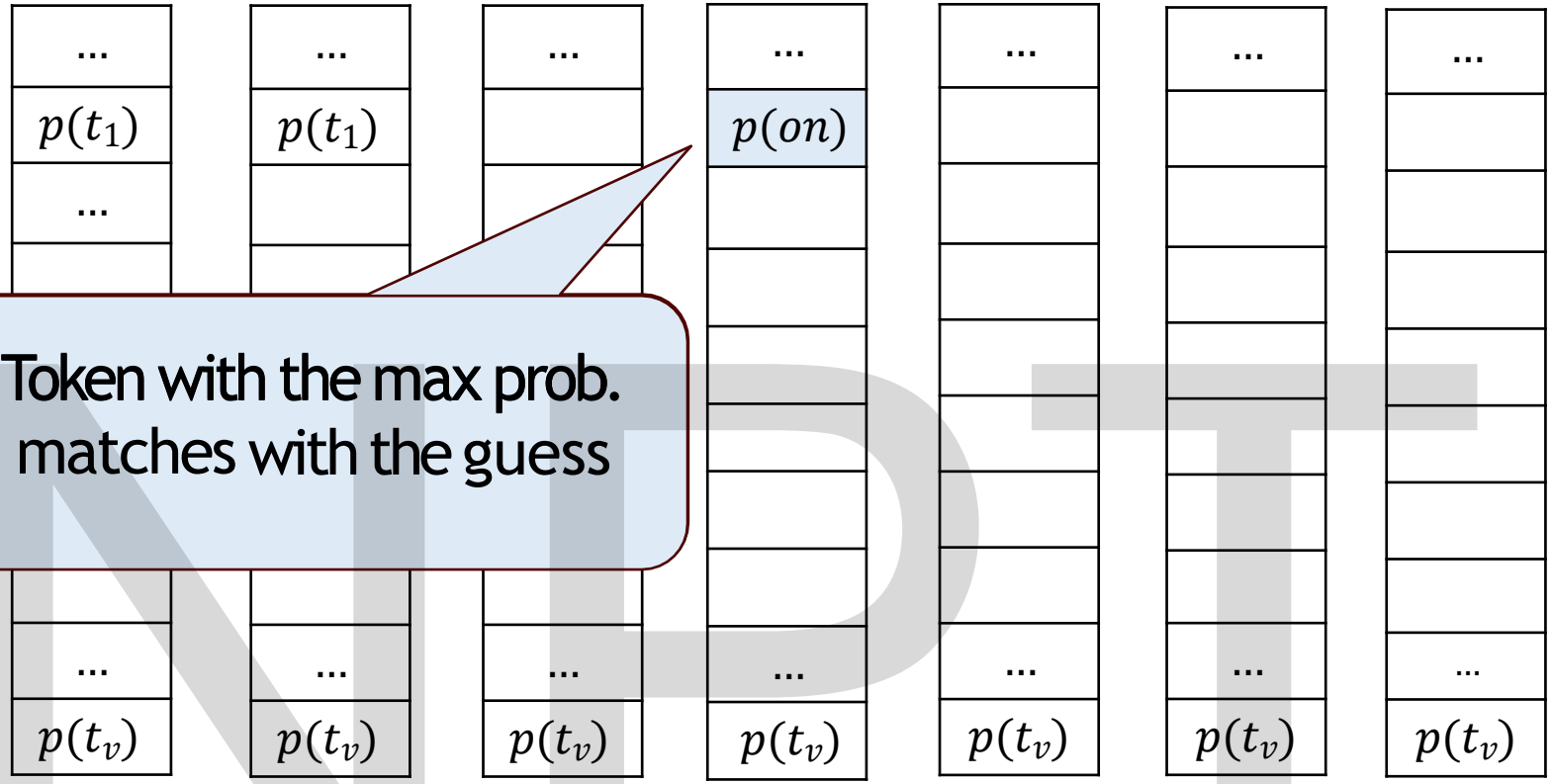


- **Input prompt:** *"The cat sat"*
- **Guess:** *"on the chair </s>"*

Focus on distribution at the last token in the prompt

Transformer based LLM (Φ)

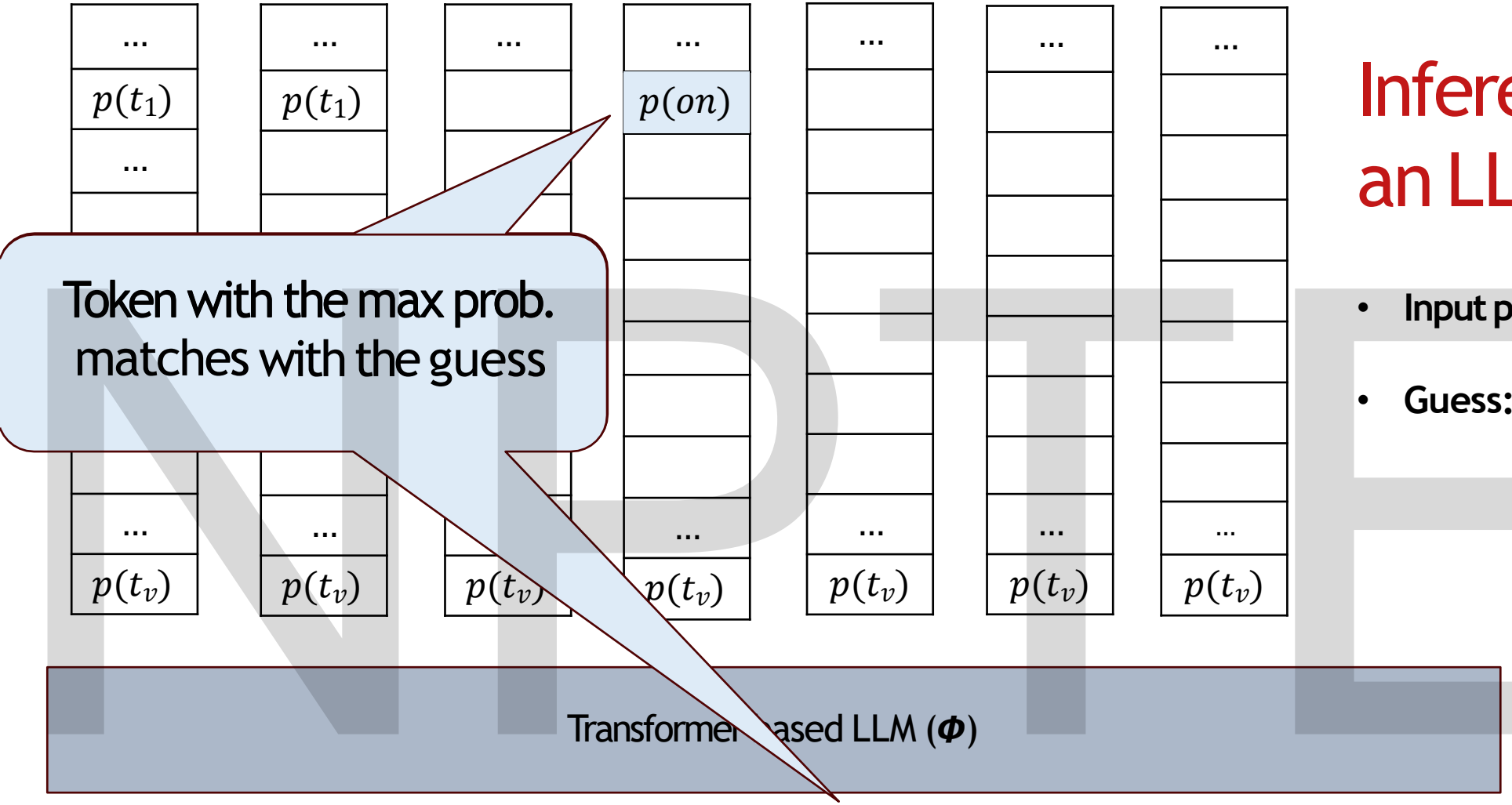
<s>	The	cat	sat	on	the	chair	</s>
0	1	2	3	4	5	6	7



Inference through an LLM

- **Input prompt:** *"The cat sat"*
- **Guess:** *"on the chair </s>"*

NPTEL

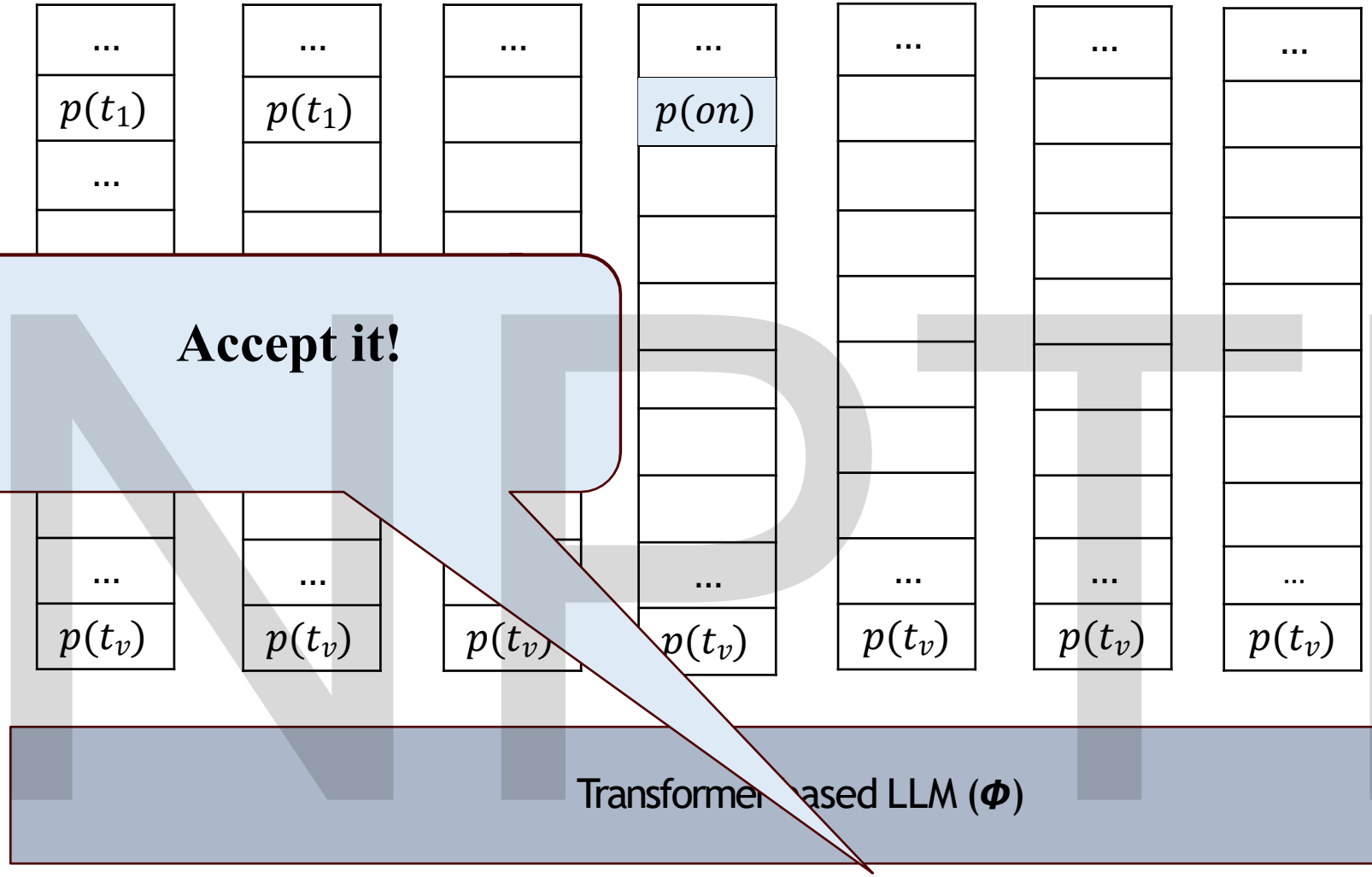


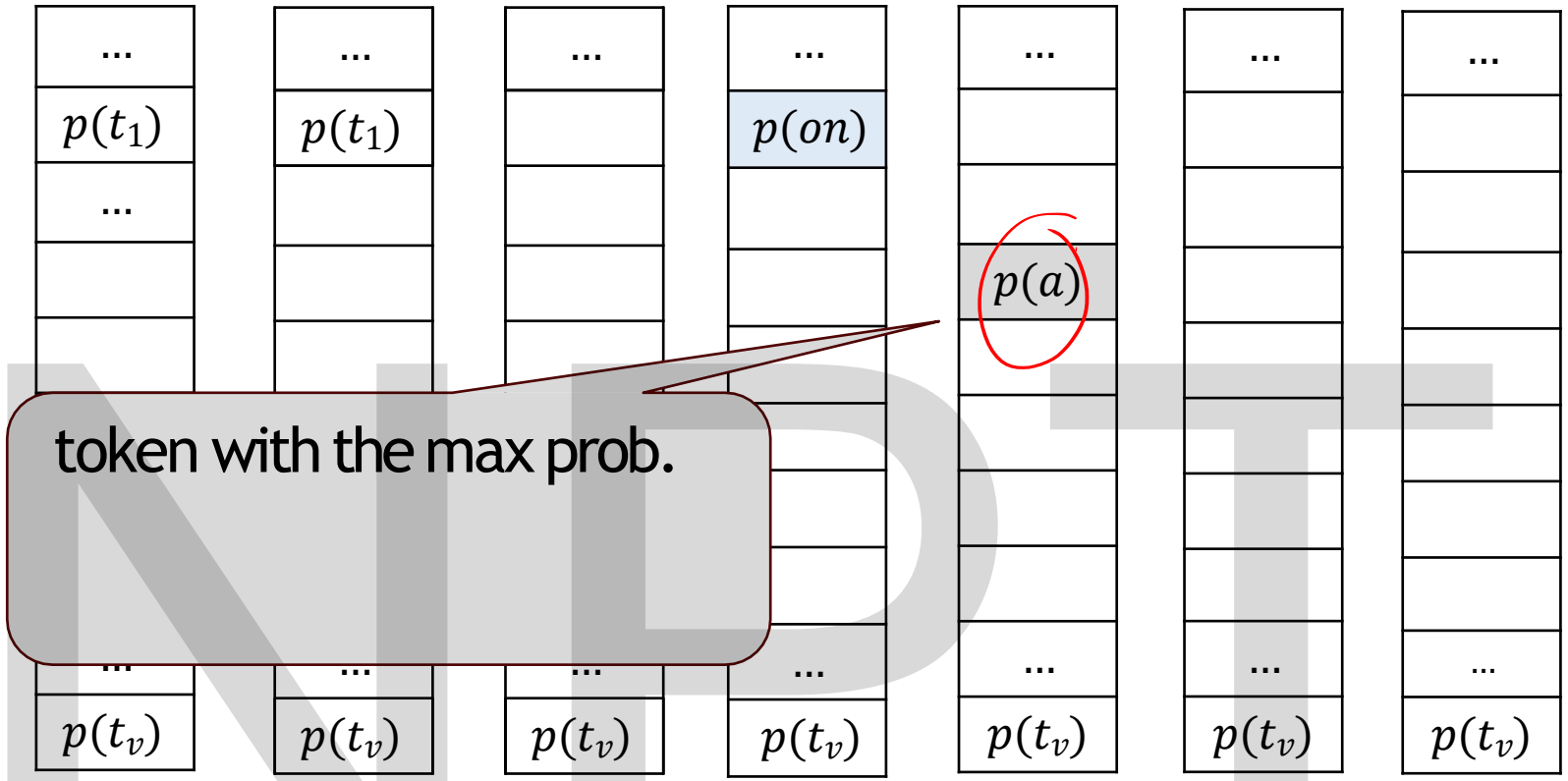
Inference through an LLM

- **Input prompt:** *"The cat sat"*
- **Guess:** *"on the chair </s>"*

Inference through an LLM

- **Input prompt:** *"The cat sat"*
- **Guess:** *"on the chair </s>"*

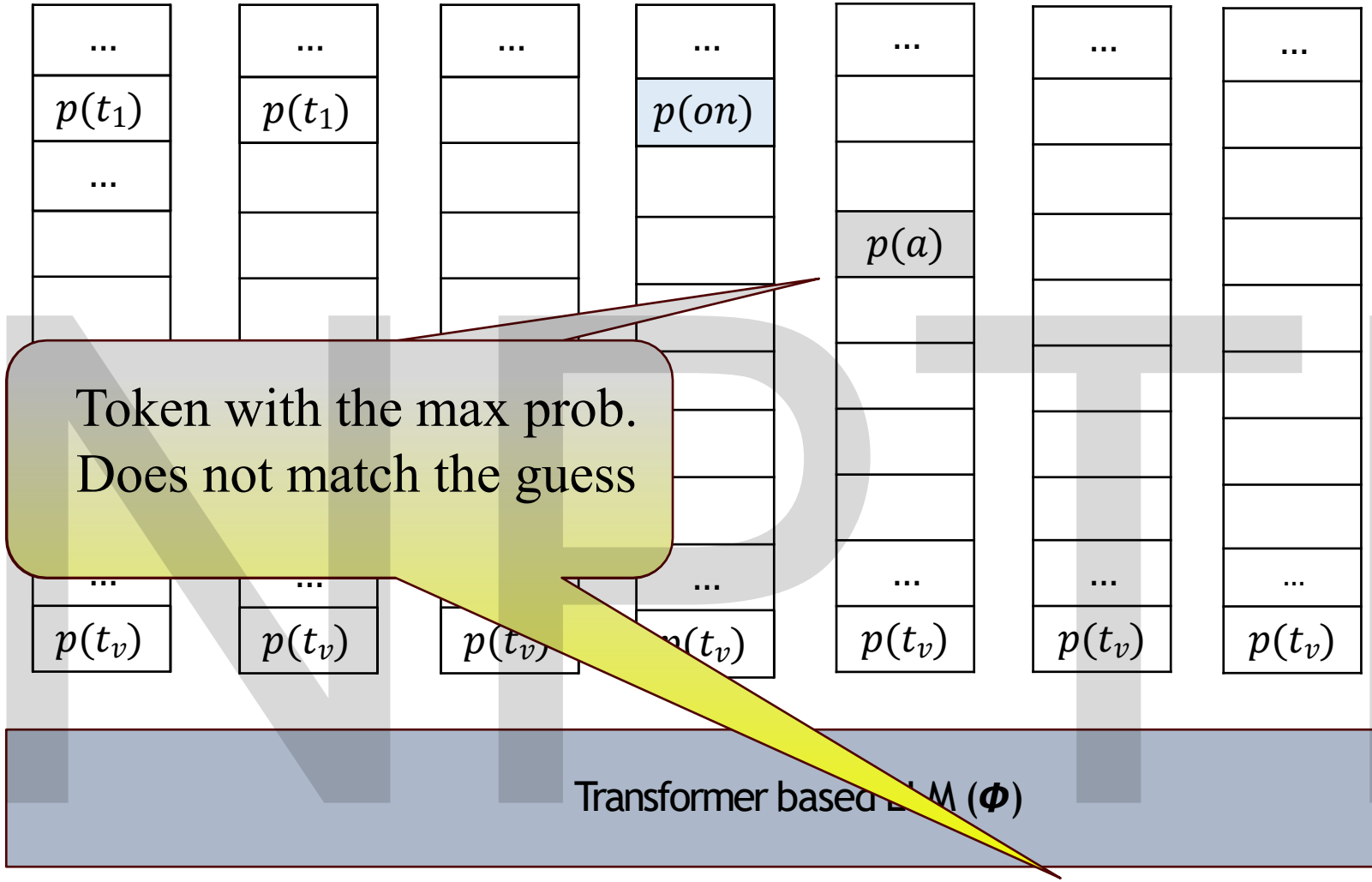




- **Input prompt:** *"The cat sat"*
- **Guess:** *"on the chair </s>"*

Transformer based LLM (Φ)

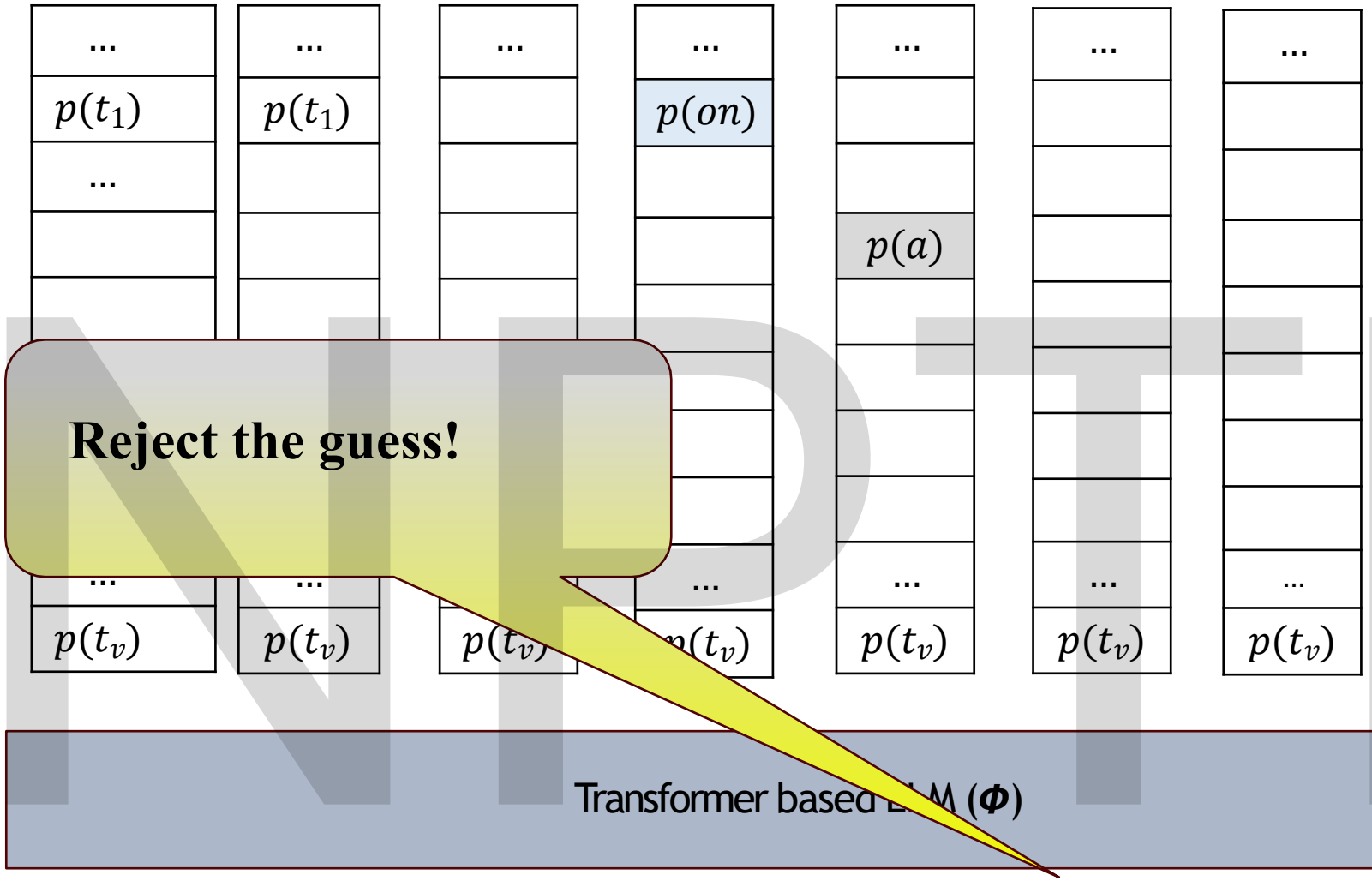
<s>	The	cat	sat	on	the	chair	</s>
0	1	2	3	4	5	6	7



Inference through an LLM

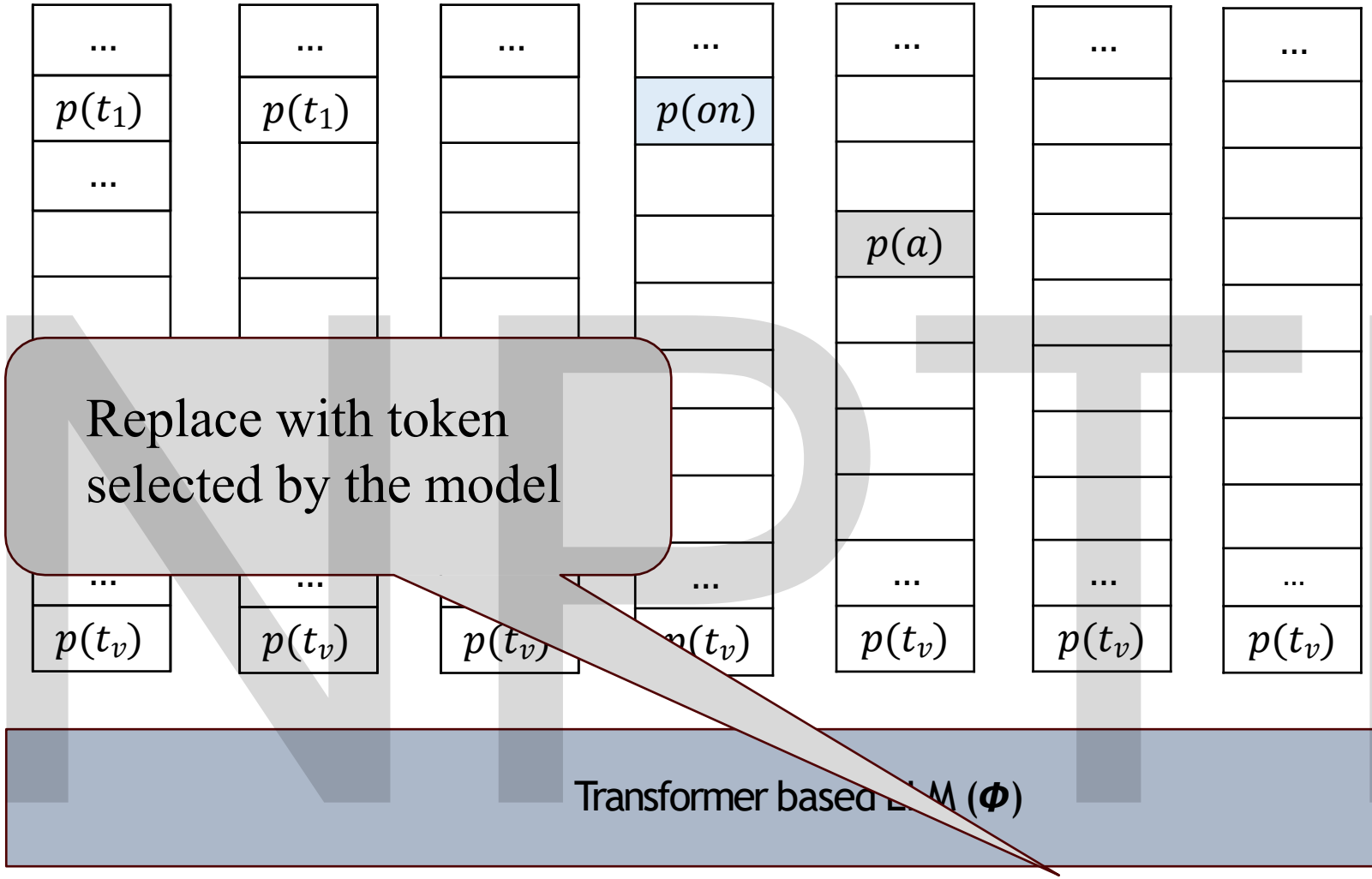
- **Input prompt:** *"The cat sat"*
- **Guess:** *"on the chair </s>"*

NPTEL



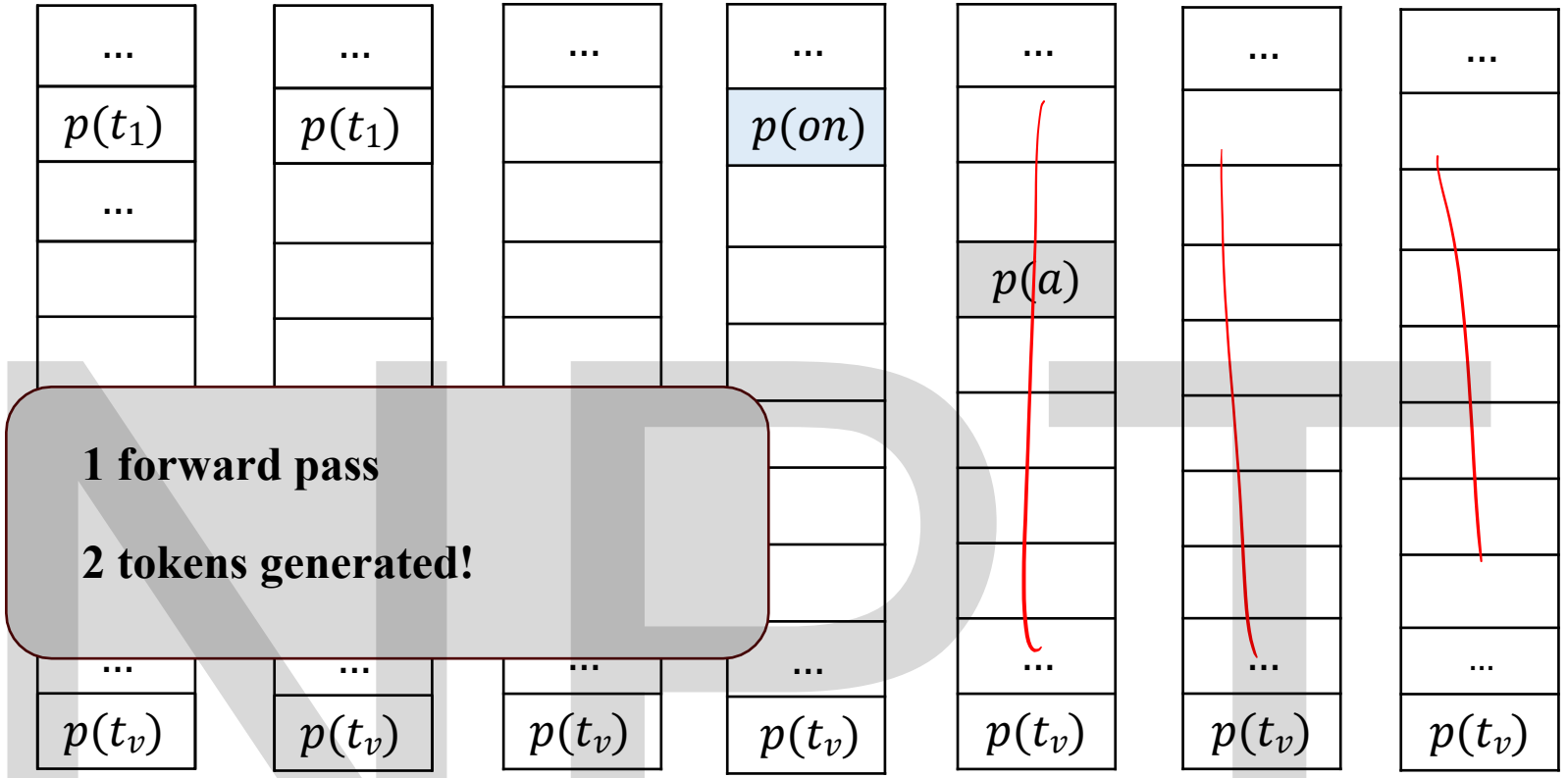
- **Input prompt:** *"The cat sat"*
- **Guess:** *"on the chair </s>"*

NPTEL



Inference through an LLM

- **Input prompt:** *"The cat sat"*
- **Guess:** *"on the chair </s>"*



1 forward pass

2 tokens generated!

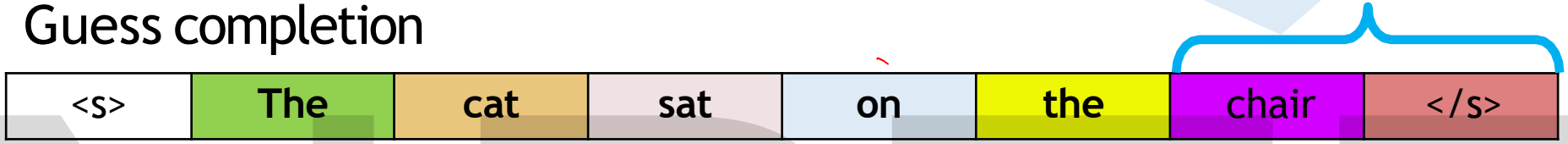
- **Input prompt:** *"The cat sat"*
- **Guess:** *"on the chair </s>"*

Transformer based LLM (Φ)

<s>	The	cat	sat	on	a	chair	</s>
0	1	2	3	4	5	6	7

Can't use rest of the completion as it was dependent on token "the" that has been rejected

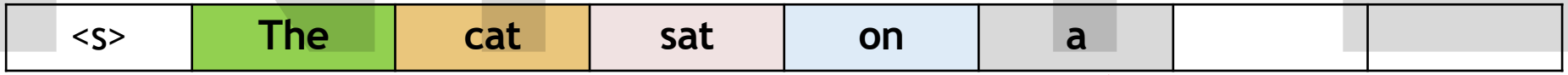
Guess completion



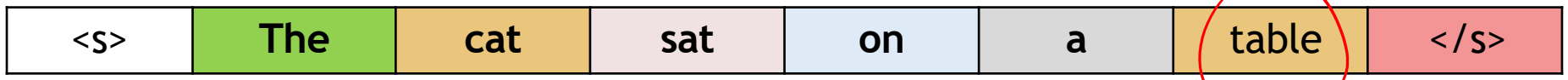
Verification by the LLM



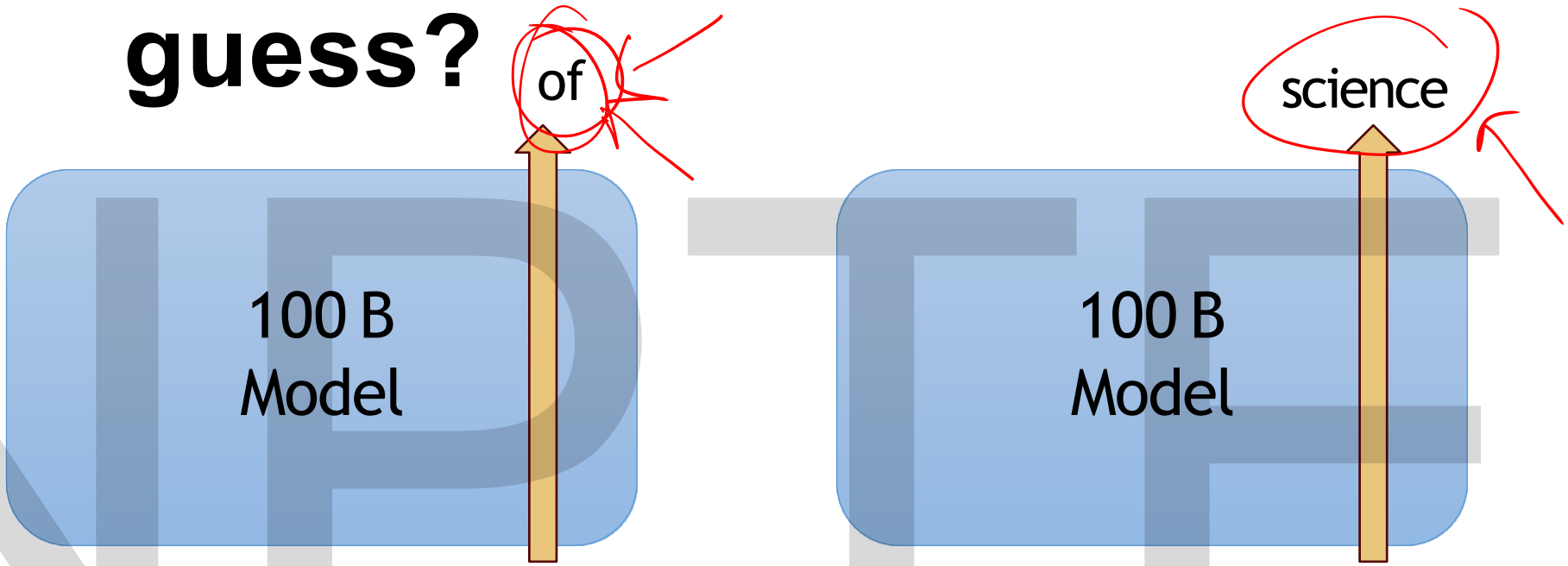
New Input Prompt



New Guess



How to make a guess?



Very easy

Difficult

Content credit: https://youtu.be/S-8yr_RibJ4?si=-u2dh3PRBwTnXBOZ

Observation

Some tokens are easier than others.



Takeaway : Use a much smaller draft model to guess easy tokens



THANK YOU

Slide Courtesy: Murali Annavaram (University of Southern California), R. Govindarajan (IISc), Arkaprava Basu (IISc), Jayashree Mohan (Microsoft Research)